

INF 574

a.k.a. CSC 51074 EP

Geometric Modeling: Digital Representation and Analysis of Shapes



Today: Fundamentals To Get Started

- Geometric Modeling – general goals
- Transformations
- Rotations:
 - Matrices,
 - Euler angles,
 - Quaternions
- Homogeneous Coordinates
- Projective Transformations



General Goals

Goal 1:

To represent (2d, 3d) shapes in a way that makes it easy to *manipulate* them.



roll

yaw

pitch

Common manipulations:

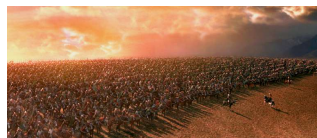
Rotation, translation, scaling, projection



General Goals

Goal 2:

To decouple the representation from the transformation. “Model once, use often”



Lord of the Rings: Return of the King

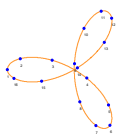


[D. James, C. Twigg]



Representation Taxonomy

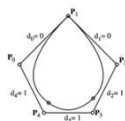
A zoo of 2D shape representations



Interpolating cubic splines



Non-interpolating Bézier curves

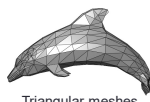


B-splines



Representation Taxonomy

A zoo of 3D shape representations



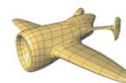
Triangular meshes



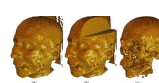
NURBS surfaces



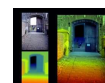
Point clouds



Quad meshes



Volumetric meshes



2D depth maps



Isometries of Euclidean Space

Thus, a rotation around the origin can be represented as a **matrix**:

$$R = \begin{pmatrix} T(e_1) & T(e_2) & \dots & T(e_n) \end{pmatrix}$$

So that $T(p)$ is computed via Rp

- where $p_i = \langle p, e_i \rangle$ encode p 's position in the chosen orthonormal basis
- ex: in 2D, $R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ $(x, y) = (r \cos \phi, r \sin \phi) \longrightarrow (x', y') = (r \cos(\phi + \theta), r \sin(\phi + \theta))$
 $(x', y') = (x \cos(\theta) - y \sin(\theta), x \sin(\theta) + y \cos(\theta))$

Note that:

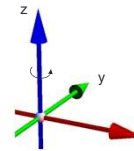
- R is orthogonal (inverse = transpose)
 - obvious from preservation of inner product, since $p^t q = p^t R^t R q$
- In 2D, R has one fixed point; in 3D, a fixed axis
 - 3D axis of rotation: eigenvector of R with eigenvalue 1
 - funny question: does the notion "axis of rotation" make sense in 4D, 5D, etc.?



Rotations

Rotation about an axis:

e.g., rotation about z -axis; all z coordinates remain the same:



Rotations

Rotation about an axis:

e.g., rotation about z -axis by θ radians:

$$\mathcal{R}_{z,\theta} : \begin{pmatrix} x \\ y \\ z \end{pmatrix} \longrightarrow \begin{pmatrix} x \cos(\theta) - y \sin(\theta) \\ x \sin(\theta) + y \cos(\theta) \\ z \end{pmatrix}$$

In matrix form:

$$\mathcal{R}_{z,\theta} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$



Rotations

Rotation about an axis:

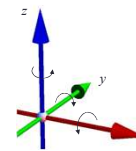
rotation about *an* axis (counterclockwise) by θ radians:

$$\mathcal{R}_{x,\theta} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix}$$

Note: difference in sign is due to right-handedness:

$$\mathcal{R}_{y,\theta} = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix}$$

$$\mathcal{R}_{z,\theta} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

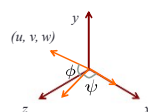


Rotations

Rotation about an *arbitrary* axis:

how to rotate about an axis $I=(u, v, w)$ by angle θ ?

- rotate about x axis to make $v=\theta$
- rotate about y axis to make $w=\theta$
- rotate about x axis by θ
- undo second rotation
- undo first rotation



$$\mathcal{R}_{(u,v,w),\theta}(P) =$$

(In matrix notation, composition is simply matrix product!)



Rotations – Take Two

Rotation about an *arbitrary* axis:

how to rotate about an axis $I=(u, v, w)$ by angle θ ?

Rodrigues' rotation formula

$$R_{(u,v,w),\theta} = I + (\sin \theta)U + (1 - \cos \theta)U^2$$

$$\text{where } U = \begin{pmatrix} 0 & -w & v \\ w & 0 & -u \\ -v & u & 0 \end{pmatrix}$$

- can be derived by noting that $Ux = I \times x$ (cross product in 3D), & observing the action of R on vectors parallel and orthogonal to I .



Rotations – Discussion

General Questions:

- Minimal number of parameters to specify a rotation?
- How to represent rotations without wasting parameters?
- Are all parameterizations equivalent?

Optimal representations: unfortunately none...

See: [Stuelpnagel, John](#), "On the parametrization of the three-dimensional rotation group." *SIAM review* 6.4 (1964): 422-430.



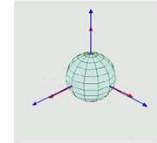
Rotations – Other Representations

Euler angles or Tait-Bryan angles:

Represent an arbitrary rotation by a combination of three rotations around the principal axes; **e.g.**:

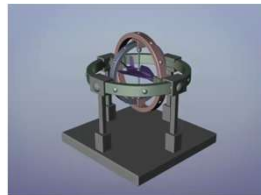
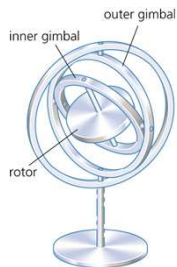
$$R = R_{x,\alpha} R_{y,\beta} R_{z,\gamma} \\ = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\alpha) & -\sin(\alpha) \\ 0 & \sin(\alpha) & \cos(\alpha) \end{pmatrix} \begin{pmatrix} \cos(\beta) & 0 & \sin(\beta) \\ 0 & 1 & 0 \\ -\sin(\beta) & 0 & \cos(\beta) \end{pmatrix} \begin{pmatrix} \cos(\gamma) & -\sin(\gamma) & 0 \\ \sin(\gamma) & \cos(\gamma) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\text{or: } R = R_{z,\alpha} R_{x,\beta} R_{z,\gamma}$$



Euler Angles and Gimbals

Euler/Tait-Bryan angles are related to Gimbal systems:



GuerillaCG on YouTube



Flaw of Euler Angles: Gimbal Lock

Gimbal lock seen in matrix notation:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\alpha) & -\sin(\alpha) \\ 0 & \sin(\alpha) & \cos(\alpha) \end{bmatrix} \begin{bmatrix} \cos(\beta) & 0 & \sin(\beta) \\ 0 & 1 & 0 \\ -\sin(\beta) & 0 & \cos(\beta) \end{bmatrix} \begin{bmatrix} \cos(\gamma) & -\sin(\gamma) & 0 \\ \sin(\gamma) & \cos(\gamma) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

If $\beta = \frac{\pi}{2}$ then:

$$R = \begin{bmatrix} 0 & 0 & 1 \\ \sin(\alpha + \gamma) & \cos(\alpha + \gamma) & 0 \\ -\cos(\alpha + \gamma) & \sin(\alpha + \gamma) & 0 \end{bmatrix}$$

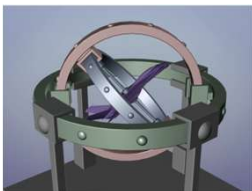
Euler angles lose degrees of freedom...

Cannot represent all continuous paths of rotations!

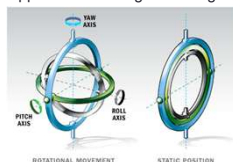


Flaw of Euler Angles: Gimbal Lock

Gimbal lock visually



Appears when 2+ gimbals align



2 degrees of freedom instead of 3



Rotations – Why Euler Fails?

Problem simple to identify

- rotation around an axis a by θ is the *same* as around $-\alpha$ by $-\theta$
- rotation around an axis a by π , is the *same* as by $-\pi$

Consequence: one **cannot** represent rotations consistently with 3 periodic parameters

- no local homeomorphism from a 3-torus to the space of rotations
- **topological mismatch**: map unable to be a *covering* map
- singularities: gimbal locks

Can be a really annoying issue, obviously!

- not always; for cameras, put gimbal lock when looking at sky/ground
- check out funny Appollo 11 joke (*I want a fourth gimbal for Christmas*)



Enter Quaternions

Quick reminder:

1. Vectors with 4 components, 3 of which are imaginary:

$$q = a + bi + cj + dk$$

2. Adding quaternions works simply elementwise

$$(a_1 + b_1i + c_1j + d_1k) + (a_2 + b_2i + c_2j + d_2k) = (a_1 + a_2) + (b_1 + b_2)i + \dots$$

3. Multiplication is defined using the rules:

$$i^2 = j^2 = k^2 = ijk = -1 \quad \text{Thus: } \begin{matrix} ij = k & jk = i & ki = j \\ ji = -k & kj = -i & ik = -j \end{matrix}$$

So one has: $q_1 q_2 = (a_1 + b_1i + c_1j + d_1k)(a_2 + b_2i + c_2j + d_2k)$

$$\begin{aligned} &= a_1 a_2 - b_1 b_2 - c_1 c_2 - d_1 d_2 \\ &+ (a_1 b_2 + b_1 a_2 + c_1 d_2 - d_1 c_2)i \\ &+ (a_1 c_2 - b_1 d_2 + c_1 a_2 + d_1 b_2)j \\ &+ (a_1 d_2 + b_1 c_2 - c_1 b_2 + d_1 a_2)k \end{aligned} \quad q \sim \begin{pmatrix} a & -b & -c & -d \\ b & a & -d & c \\ c & d & a & -b \\ d & -c & b & a \end{pmatrix}$$

Can use matrix form instead...

Enter Quaternions

Quick reminder:

1. Vectors with 4 components, 3 of which are imaginary:

$$q = a + bi + cj + dk$$

2. Adding quaternions works simply elementwise

$$(a_1 + b_1i + c_1j + d_1k) + (a_2 + b_2i + c_2j + d_2k) = (a_1 + a_2) + (b_1 + b_2)i + \dots$$

3. Multiplication is defined using the rules:

$$i^2 = j^2 = k^2 = ijk = -1 \quad \text{Thus: } \begin{matrix} ij = k & jk = i & ki = j \\ ji = -k & kj = -i & ik = -j \end{matrix}$$

Another convenient operation: **conjugate**

$$q^* = a - bi - cj - dk$$

$$\|q\|^2 = qq^* = q^* q = a^2 + b^2 + c^2 + d^2$$

$$q^{-1} = \frac{q^*}{\|q\|^2}$$

Rotations – Using Quaternions

Let's represent *real vectors* as pure imaginary quaternions:

$$v = 0 + xi + yj + zk$$

To rotate v around a (unit) axis (u_x, u_y, u_z) by angle θ , define:

$$q = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(u_x i + u_y j + u_z k)$$

Then rotation acts through conjugation:

$$v' = qvq^*$$

Check it by showing that v' is pure imaginary, has the same norm as v , that u is invariant, and that the angle of rotation is θ

Rotations – Benefits of Quaternions

Seeing rotations as conjugation of unit quaternions is nice

- every rotation is represented by a unit quaternion (or its opposite)

- this provides a *double cover* of the space of rotations

- isomorphic to $SU(2)$, closely related to spinors in physics
- no singularity to worry about!

- operations on rotations often simple!

- ex; interpolation between two rotations through "slerp" (spherical linear interpolation)

From two unit quaternion such that $q_0 \cdot q_1 = \frac{1}{2}(q_0 q_1^* + q_0^* q_1) \geq 0$

- if the dot product is negative, just negate one of them

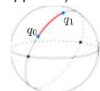
we can define an **in-between quaternion** through $q(t) = \text{slerp}(q_0, q_1, t)$

$$\text{with } q(t) = (q_1 q_0^{-1})^t q_0$$

where for a quaternion $p = a + \mathbf{v}$, $p^t = \exp(t \ln p)$

$$\exp(p) = \exp(a) \left(\cos \|\mathbf{v}\| + \frac{\mathbf{v}}{\|\mathbf{v}\|} \sin \|\mathbf{v}\| \right) \quad \ln(p) = \ln \|p\| + \frac{\mathbf{v}}{\|\mathbf{v}\|} \arccos \frac{a}{\|p\|}$$

- averages of rotations too with a bit of work



Other Transformations

Scaling:

- uniform:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{pmatrix} s x \\ s y \\ s z \end{pmatrix}$$

In matrix form:

$$S_s = \begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & s \end{pmatrix}$$

- non-uniform:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{pmatrix} s_x x \\ s_y y \\ s_z z \end{pmatrix}$$

In matrix form:

$$S_{s_x, s_y, s_z} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{pmatrix}$$

- scaling along different axes: align by rotation, scale, undo alignment...

Other Transformations

Translation:

$$\mathcal{T}_{t_x, t_y, t_z}(x, y, z) : \begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \end{pmatrix}$$

- Inverse trivial! just the opposite translation

- Problem though... not a linear operation!

- i.e., does not preserve the origin

- Translations cannot be represented through matrix multiplication**

- major issue... but with a simple solution

Affine Homogeneous Coordinates

Let's modify the coordinates by adding an extra dimension:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

▪ nothing breaks in prior transformation: just add an extra row & column

ex:

$$\mathcal{R}_{\theta} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \cos(\theta) - y \sin(\theta) \\ x \sin(\theta) + y \cos(\theta) \\ z \\ 1 \end{pmatrix}$$

▪ now, translation becomes linear

$$\mathcal{T}_{t_x, t_y, t_z} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{pmatrix}$$

▪ any affine transformation in 3D is a linear transformation in 4D

Projections

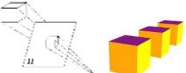
We also need to **project** a 3D model onto the 2D screen

Two most common projections

▪ Orthographic:



▪ Perspective:



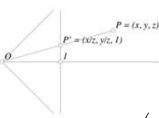
▪ Check paintings in museums...

Projections in Matrix Form

Good news, 4D coordinates is just what we need here too

▪ Orthographic projection: $\begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{pmatrix} x \\ y \\ 0 \end{pmatrix}$ $\mathcal{P}_0 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

▪ Perspective projection: $\begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{pmatrix} x/z \\ y/z \\ 0 \end{pmatrix}$ **Ouch...**



$$\mathcal{P}_p \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ 0 \\ z \end{pmatrix} = \begin{pmatrix} x/z \\ y/z \\ 0 \\ 1 \end{pmatrix}$$

▪ we simply used: $\begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{pmatrix} x/w \\ y/w \\ z/w \\ 1 \end{pmatrix}$ **hence the name: "homogeneous coordinates"**

Fancier Transformations

Tapering

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & f(x) & 0 & 0 \\ 0 & 0 & f(x) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



Twisting



Bending

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & f(y) & g(y) & 0 \\ 0 & h(y) & k(y) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



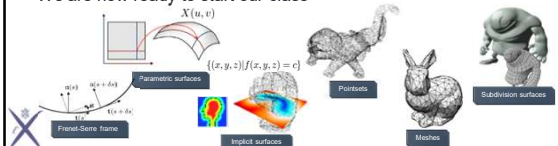
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta(y)) & 0 & \sin(\theta(y)) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta(y)) & 0 & \cos(\theta(y)) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Summary

Geometric modeling is about **shape representation & manipulation**

- all existing representations involve points in one way or another.
- In Euclidean world, isometric transformations are translations/rotations
 - other relevant transformations include scaling, projection...
- representation of rotations: rotation matrices, quaternions...
- all conveniently represented w/ matrices; composition = matrix multiplication.
- homogeneous coordinates "linearize" most useful transformations

We are now ready to start our class



Questions?

